Phase 2 developer handoff: AI gateway

This release adds a bounded local-model evidence-selection workflow. It implements the first local adapter and enforcement path; it does not enable external providers, generated patches, code execution, deployment or a general agent tool loop.

Structure and contracts

Module	Responsibility
`backend/app/ai/models.py`	Tenant-owned immutable provider registrations and metadata-only invocation records
`ai/schemas.py`	Strict provider, provider-state and source-range query requests
`ai/filters.py`	Offline detect-secrets adapter, local PII patterns, normalization and injection checks
`ai/providers.py`	Adapter protocol and the only model HTTP transport; Ollama implementation
`ai/gateway.py`	Server-built context, live permission/policy checks, budgets, audit-before-dispatch and validated output
`ai/routes.py`	Registration/connectivity/state/history/source-query HTTP APIs
`frontend/src/security/AIConsole.tsx`	Provider/policy controls, source-range form and invocation history
`0004_ai_gateway.py`	Additive, frozen migration for ai_providers and ai_invocations

Static assistant selection is integrated in services/assistant.py; runtime selection is integrated after the existing deterministic evidence builder in runtime/routes.py. No model replaces analysis algorithms or alters stored snapshots. The legacy OllamaProvider remains a disabled guard; all operational model calls use the gateway.

Dispatch flow

1. Existing request authentication and tenant-scoped parent lookup authorize access to the selected analysis/session/repository. No client-provided fact list, model endpoint or context envelope is accepted by an assistant endpoint.
2. The gateway reloads active principal permissions, latest workspace policy and enabled tenant-owned provider. MODEL_USE_LOCAL plus ANALYSIS_READ, RUNTIME_READ or SOURCE_READ is mandatory. Source requests additionally require the exact commit, current explicit grant and enabled source policy.
3. The server selects at most 12 facts. Source requests go through the Code Security Gateway, then select up to ten explicit ranges, 80 lines each and 200 total lines. Unselected content is omitted from the model envelope.
4. Normalize text, block suspected secrets/instructions and redact or block known sensitive identifiers. Scan the exact serialized outgoing question/evidence envelope again. Opaque e0/e1 IDs replace actual evidence IDs, paths and URLs on the model wire.
5. Enforce serialized UTF-8 byte size (default 16 KiB; maximum 32 KiB). Treat context as RESTRICTED_LOCAL_ONLY even after filtering. Build a SHA-256 context manifest; a prepared envelope expires after 60 seconds before authorization.
6. Recheck policy revision, provider identity and source scope; reserve the workspace hourly attempt budget under the security lock and durably commit REQUESTED audit before provider I/O. No automatic external retry/fallback exists.
7. Recheck the registered installed model digest, then send the fixed system instruction and minimized user envelope through the local adapter. Bound provider time, raw response bytes, generated tokens and context window. No model tools are supplied.
8. Require a completed assistant response without tools/images and strict JSON containing only a unique list of allowed citation IDs. Filter the output; reject malformed JSON, invented IDs, additional generated prose/commands, sensitive output or empty selection.

9. Recheck current principal/policy/provider/source scope after inference. Commit final audit before releasing validated evidence text. On rejection, static/runtime assistants use existing deterministic facts; source queries return no AI-selected source. Audit-storage failure prevents privileged output.

In-memory context never becomes a database prompt record or reusable client capability. Client replay cannot submit an old envelope. Registrations cannot change endpoint/model/digest through the API; updating a model means registering a new identity and changing policy revision.

Filters and limitations

Pinned dependency: detect-secrets 1.5.0. The adapter calls offline analyze_string methods, never network-based credential verification. Included detectors cover keywords, private keys, AWS-like keys, JWTs, basic-auth URLs and high-entropy quoted strings. Entropy candidates are tested against the upstream entropy threshold before being classified. Supplementary patterns cover bearer credentials and key/password/token assignments.

PII patterns recognize emails, international phone-like numbers, payment-number-like strings and US SSN formatting. PII can REDACT or BLOCK; suspected secrets always BLOCK. Filtering records categories/reason codes, never matched values. Unicode normalization and selected injection/control patterns add rejection signals, but the primary defense against model instructions is strict citation-only output and no tools/code execution.

These detectors have false positives and false negatives. They do not identify all names, addresses, medical records, embedded business secrets or adversarial encodings. Unknown data remains restricted to explicitly allowed local processing; filtering does not make content eligible for export. A future detector/classification adapter can extend this boundary without changing assistant retrieval.

Provider transport and network boundary

Only LOCAL_MODEL/OLLAMA is operational. ProviderAdapter is the extension contract; remote destination classes are rejected, including an EXTERNAL_APPROVED_MODEL label. The server allowlist must include the exact endpoint, which must also parse as HTTP at literal 127.0.0.1, with a port and no credentials/path/query/fragment. ArchIntel port 8000 is forbidden. There is no DNS resolution or hostname-based local trust.

HTTPX uses trust_env=False, follow_redirects=False, a three-second connect timeout and bounded operation timeout (default 60 seconds, configurable 1-120). Redirects, unexpected compression and responses larger than 65,536 raw bytes are rejected. Model presence and a pinned digest are checked via /api/tags; cloud-named/remote/zero-size entries are rejected. The adapter calls /api/chat with JSON schema, temperature zero, num_predict budget, num_ctx=4096 and keep_alive=0.

The fixed schema contains only generated opaque IDs and the system instruction is fixed code. The exact serialized user envelope is rescanned; no source, question, URL or user-supplied instruction is interpolated into the system prompt. Returned prose is never trusted as an architectural fact.

This is application transport enforcement, not a host firewall/container sandbox. Trusted operators must control the local model service, cloud capabilities, its own logs/cache, and network egress independently. No air-gap certification or OS-level egress guarantee is claimed. The downloaded model is local, but offline software distribution and model provenance governance remain deployment work.

Official interfaces consulted: Ollama chat API, installed models API, detect-secrets. Adapter/filter behavior is verified by local tests, not inferred solely from these interfaces.

Policy and permission evolution

Policy JSON adds model_enabled=False, model_provider_id, model_context_bytes=16384, model_output_tokens=128, model_calls_per_hour=30, sensitive_data=REDACT and deployment_profile=LOCAL_PRIVATE. Existing revisions are not rewritten: missing fields read as safe defaults. Enablement validates a provider in the authenticated workspace. Existing external_transfer=DENY and telemetry=DISABLED remain fixed. Revision checks prevent stale policy saves.

MODEL_USE_LOCAL is an explicit new permission. Existing principals are not migrated to receive it. New full-permission administrators receive the current permission set at creation. POST security/principals/{id}/permissions lets a security administrator grant/revoke permissions; service principals still cannot receive SECURITY_ADMIN and the current administrator/installation owner cannot lose security administration through this API.

The hourly limit counts reserved model attempts across a workspace, including failed attempts after dispatch authorization. Capacity reservation and policy writes use the single-process security RLock. Network I/O occurs outside that lock, so administrators can revoke policy during inference. An already authorized/sent request cannot be unsent; output is suppressed if the post-inference authorization check fails. Multiple API processes remain unsupported until distributed authorization/budget fencing exists.

Audit and retention

ai_invocations stores request identity, principal/provider/resource IDs, time, outcome/reason and metadata. REQUESTED state is updated to ALLOW or REJECTED; append-only security audit retains transition history with the same request ID. Denied requests are recorded without issuing model HTTP. Metadata includes policy/filter versions, fixed purpose, classification, exact envelope hash/size, selected/omitted counts, redaction categories, model digest, duration and bounded usage counts. Local monetary cost fields are null because hardware cost is not measured.

No prompt, raw source snippet, response text, user question, raw match, provider exception string or provider credential is stored in these records. Model endpoints need no secret in this local adapter. Future remote adapters require customer-managed secret references, never plain API keys in provider/config export. Raw-output persistence is not an extension point.

Security audit has the Phase 1 database triggers/hash chain. Invocation rows are operational summaries, not an independent immutable ledger. Audit external anchoring, retention policies for security metadata and centralized customer key management remain future work.

API and migration

All new model APIs are under `/api/v1/security/ai`:

Endpoint	Permission / behavior
GET/POST providers	SECURITY_ADMIN; list or register installed pinned local model
POST providers/{id}/state	SECURITY_ADMIN; enable/disable
POST providers/{id}/check	SECURITY_ADMIN; audited installed-model connectivity check
GET invocations	AUDIT_READ; latest 100 tenant-scoped metadata rows
POST source-query	SOURCE_READ plus gateway MODEL_USE_LOCAL/policy/grant checks

Existing assistant endpoints return their existing response shape with updated mode/limitations; runtime also includes gateway metadata. Unsupported questions continue to abstain without a model request. Source queries include reason and request ID even on deterministic/no-source fallback.

Migration 0004 adds two tables and tenant indexes without modifying business records, existing identity grants or policy revisions. Migration 0003 still depends on Phase 1 model table definitions, which must not be changed incompatibly. Back up deployments before applying future migrations; a local row snapshot is not a tested disaster recovery procedure.

Configuration: AI_ALLOWED_ENDPOINTS defaults to ["http://127.0.0.1:11434"], AI_TIMEOUT_SECONDS defaults to 60. Legacy LLM_URL/LLM_MODEL values are not copied into trusted registrations. The API server does not download models, start model processes or manage external credentials. The prepared development machine has qwen2.5:0.5b installed for integration tests.

Reproduce verification

```
$phaseTests = '.data/phase2-check-' + [guid]::NewGuid().ToString('N')
$env:ARCHINTEL_RUN_MODEL_TESTS = '1'
$env:ARCHINTEL_RUN_SCANNER_TESTS = '1'
.\.runtime\python\tools\python.exe -m pytest backend/tests -q -p no:cacheprovider --basetemp=$phaseTests
Set-Location frontend
npm.cmd test
npm.cmd run build
npx.cmd playwright test --config playwright.security.config.ts
```

Model tests require Ollama plus the installed qwen2.5:0.5b tag. Ordinary security tests use a constrained test adapter or local HTTP fixture; real integration tests use the real model with synthetic evidence. Browser checks use a unique temporary SQLite installation on port 18001 and never consume/reset the actual owner account. See PHASE-2-VERIFICATION.md for actual outcomes and any pending validation.

Before adding generative advice, remote models, source transformations or agents, extend policy/filter/response contracts and negative tests first. Phase 3 runtime foundation is the next user verification gate, not part of this phase.

For the optional real PostgreSQL integration, set ARCHINTEL_RUN_POSTGRES_TESTS=1 and run test_postgres_model_gateway_isolated_schema in backend/tests/test_ai_gateway.py. It creates a unique test-owned schema on the configured local PostgreSQL server, exercises actual model/audit/budget behavior, then drops only that schema. The existing application schema and identity are untouched. This test also requires the local model.